FACE: A Mobile Face Recognition Device

Aneesha Nitin Gujral[1], Anselm Mmaju[2], Frank Laubach[3], Kshama Dhaduti[4], Manas Reddi Katam Reddi[5], Rohit Hiraman Rajput[6]

Department of Electrical and Computer Engineering, College of Engineering
Northeastern University, Boston, MA, USA

[1] gujral.a@northeastern.edu
[2] mmaju.a@northeastern.edu
[3] laubach.f@northeastern.edu
[4] dhaduti.k@northeastern.edu
[5] katamreddi.m@northeastern.edu
[6] rajput.ro@northeastern.edu

Abstract— **This paper details our product: FACE which is developed to help its user recognize faces. FACE is targeted towards those suffering from prosopagnosia, a common disability that prevents those it afflicts from recognizing faces. It is more serious than plain bad memory, and there are currently no widespread assistive technologies to alleviate its affects. To develop our product, we took the basic idea of Augmented Reality glasses, and specialized it solely towards the goal of facial recognition. Development also focused on ease of use and affordability. Our resulting product was successfully able to recognize a target's face and conveyed the identity of the target to the user through an android application. Further development could allow us to produce our product at a wide market scale, and address prosopagnosia, an issue that is now mostly unnoticed, head-on.**

Keywords— **prosopagnosia, Assistive technology, augmented reality glasses, android application**

I. INTRODUCTION

The original idea for the FACE project was inspired by a difficulty experienced by a significant portion of the population: meeting a new person, then soon after forgetting their name. This common scenario takes place many times throughout the course of a year; attending business conferences, going to a wedding, meeting new people at a workplace or in school, and getting to know people at various other gatherings. While this may seem like a minor inconvenience, easily solvable by nametags or mental training, the problem is much more widespread across the world than it would first appear. After further researching the problem, we came across several articles discussing variation in the ability to remember faces and names between different individuals. While some people have good facial memory, and others have poor facial memory, there also exists a portion of the population that has a complete inability to remember faces. Several research articles and survey surmise that around 2% of the population experiences total face blindness, a disability called prosopagnosia. While memorization techniques may work for most people and a nametag might be given to conference members, these are not universal solutions. An individual will not wear a nametag while roaming around in public, and memorization techniques require time and focus. One suffering from prosopagnosia has difficulty remembering faces all the time, not just at specific events. Further compounding the issue is the current pandemic and the resultant widespread use of facemasks, making facial identification a greater challenge even to the general public.

Therefore, we decided to design a technological solution to alleviate the effects of face blindness, poor name recall, and masking. The function of our prototype FACE is simple, it recognizes the target person, and conveys the name of the detected person to the user. Additionally, our prototype should be intuitive, unobtrusive, reliable, cost-effective, and usable in multiple scenarios. Our proposal was designed with these requirements in mind, and we believe it effectively meets them.

Our product is composed of three basic parts: the camera, the microprocessor, and a phone application. The camera is clipped on to the user's glasses and provides a continuous live video feed to the microprocessor. The microprocessor runs code built off the OpenCV framework to analyze the video feed and identify faces. If a face can be matched to the microprocessor's pre-built face database, the identity will then be sent to a web server, and then the respective person's name would be given as an output to an app installed on the user's phone. FACE has two basic modes that can be toggled between via a button found on the hip unit; continuous and triggered. In continuous mode, the microprocessor code constantly searches for faces, and sends any matched names to be displayed in the phone app. In triggered mode, the microprocessor stays idle unless another button is pressed, after which the code begins to search for a face. After a face is found in triggered mode, the results are displayed on the phone app, and the code returns to idle mode. Further specifications are detailed in the following report.

The report is broken up into subsections detailing the different aspects of our project. The next section, section II, is the Problem Statement, which provides further information on prosopagnosia and its incidence and clinical description. Afterwards, in section III Literature Review, we discuss existing technologies that may compete with our proposal. In section IV, Algorithm, we discuss our facial recognition algorithm. Section V Pseudocode provides an overview of our

project coding. The flowchart is presented in the next section, VI Flowchart. In VII, Components, a breakdown and overview of the constituent parts of our project is given. Section VIII System Architecture and Section IX Software implementation discuss the topics given by their respective titles. Section X Risk Mitigation discusses problems we foresaw in our project, and the steps we took to mitigate them. Section XI Limitations discusses the limitations of our final product. Next, in XII Accuracy Analysis, we test the accuracy of FACE. Lastly, section XIII Conclusion concludes the paper.

II. PROBLEM STATEMENT

A. People suffering from Prosopagnosia or face blindness

Prosopagnosia, also called face blindness, is a cognitive disorder of face perception in which the ability to recognize familiar faces, including one's own face (self-recognition), is impaired, while other aspects of visual processing (e.g., object discrimination) and intellectual functioning (e.g., decision-making) remain intact. The term originally referred to a condition following acute brain damage (acquired prosopagnosia), but a congenital or developmental form of the disorder also exists, with a prevalence rate of 2.5%.[1] The specific brain area usually associated with prosopagnosia is the fusiform gyrus, which activates specifically in response to faces. The functionality of the fusiform gyrus allows most people to recognize faces in more detail than they do similarly complex inanimate objects. For those with prosopagnosia, the new method for recognizing faces depends on the less sensitive object-recognition system. The right hemisphere fusiform gyrus is more often involved in familiar face recognition than the left. It remains unclear whether the fusiform gyrus is only specific for the recognition of human faces or if it is also involved in highly trained visual stimuli.

There are 2 types of prosopagnosia:

1) *Developmental prosopagnosia* – where a person has prosopagnosia without having brain damage.
2) *Acquired prosopagnosia* – where a person develops prosopagnosia after brain damage, often following a stroke or head injury.

In the past, most cases of prosopagnosia were thought to occur after a brain injury (acquired prosopagnosia).

Prosopagnosia has a high incidence (particularly DP) and can significantly impair social engagement and everyday functioning.[2] Currently there are no widely accepted treatments and instead, prosopagnosics are commonly left to learn how to recognize individuals through their own process of trial-and-error with alternative strategies (e.g., voice, gait, clothing, etc.).[3]

Several studies have indicated that DP affects ∼2% of the population, and it often runs in families.[4]

B. People who have difficulty matching names to faces

Some people find it hard to recall the names of people they have previously met when looking at their faces. The problem is growing more common in the younger generations, due to decreased facial memory or focus. As a result of this problem, affected people have a tough time talking to others, worrying about offending others by failing to recognize them. This results in low confidence in public settings and may hamper their personal and professional relationships.

III. LITERATURE REVIEW

There are several products out there termed “smart glasses” many of which may be likened to what we have designed. However, there remain significant differences between our design and the many products out there. While our design was specifically aimed at aiding people suffering from prosopagnosia and people who have difficulty remembering names when looking at faces, a lot of the “smart glasses” out there are just smart phone extension. Those glasses are useful and smart, but they can’t recognize faces, simply because they were not produced or designed to do so.

Some of the popular smart glasses out there include:

1. The proposed apple AR glasses, whose rumored capabilities include being able to synchronize with iPhone and apple computers, and being able to display emails, text, maps, games, and social media on an eyeglass screen. It will also be able to adjust to a person’s eyesight, serving the function of prescription glasses without the need for prescription lenses.
2. Google Glass, which function similarly to apple’s proposal.
3. Ghost Pacer, which is the world’s first mixed reality headset, designed specifically by runners for runners. The glasses are aimed at helping runners by creating and projecting a holographic avatar that helps the user. The avatar appears to compete with the user during a running routine, providing motivation. The product also saves the daily workout data of the user.
4. The Toshiba Dyna Edge AR Smart Glasses Solution is also like the Apple Glasses and Google Glass. It is operated using voice and touch tech. Apart from being a smart phone extension, it is also used in factories to help workers scan bar codes and communicate with each other.

The list can go on and on with different glasses for slightly different purposes. But there are two glasses we found to be quite similar to what we designed, with just a few differences in motive; the ORCAM My-eye and VUZIX

A. ORCAM

Though it is not solely used for facial recognition[13], it is an assistive technology for the blind and visually impaired. The glasses can recognize faces and say their names into a speaker close to the user’s ear. The glasses also assist users in reading books by using text to audio technology. They take input via swipe and touch patterns on specific areas on the glass frame, and respond to voice commands triggered by simply saying the phrase “Hey Orcam”. Apart from recognizing faces, the glasses can also identify products by scanning bar codes. The device also has a speaker close to the ear of the user, which it uses to output speech saying the name of the person in front of the user. The glasses also can save faces into a database whenever the user wants, all the user needs to do is put a finger on the device and scan.

B. VUZIX

The product company works in partnership with NNTC[14] for software and algorithm. The glasses are mainly for law

enforcement and are widely used by the Dubai police. It matches faces against a database stored on a wearable computer with a headset. The glasses can locate 15 faces per frame in a second and have a database capacity of a million faces without cloud connection. The glasses work best when they have a specific target to search for, explaining their utility to the police. A criminal's face can be selected, and the glasses scan for the face till that person is caught. Unlike Orcam, Vuzix cannot store new faces to the database while in operation.

The glasses have a touchpad for swipe and touch interactions, have a built-in mic for input, and a speaker for output. They connect to Bluetooth and cost about $1000.

IV. ALGORITHM

For our face recognition algorithm, we extensively used the OpenCV framework and its modules. OpenCV (Open-Source Computer Vision Library) is a library of programming functions aimed at real-time computer vision [1]. For this device we used the below steps to perform our face recognition:

1) Gather data and annotation
2) Face detection in images
3) Compute 128-d face embeddings to quantify a face
4) Train a Support Vector Machine (SVM) on top of the embeddings
5) Recognize faces in images and video streams

A. Gather data and annotation

For the data gathering, we collected 80 images from each person. This was done with the same camera to maintain image consistency and format and achieve good efficiency in face recognition. During each data-gathering session we gathered photos with the target exhibiting a variety of behavior, e.g., photos were taken with the target face tilted or angled, and some photos taken with masks.

To organize our data, we created a filter in our data gathering code so that captured images will be placed in a folder with the target person's name, allowing us to use the folder name as labeling in our model. This was done instead of annotating each individual image, which would be a less efficient process. Masked and unmasked photos were sorted into separate folders, with the masked photo folder being denotated with "targetname_mask." This helped increase accuracy in recognizing faces.

B. Face detection in images

The preliminary process in the face detection algorithm is to find the faces (location and size) in an image and extract them to be analyzed by the rest of the face detection algorithm. To detect faces in the images inside the dataset for this project we used OpenCV's pretrained caffe-based deep learning face detector model.

This face detector is based on the Single Shot Detector (SSD) framework with a ResNet base network.

C. Face Embedding extraction

To extract face embeddings, we used Facenet, a deep convolutional network model from OpenCV.

FaceNet maps data from facial images into a compact Euclidean space. Distance in this space corresponds directly to facial similarity. By using standard FaceNet techniques and FaceNet embeddings as feature vectors after this space has been produced, tasks such as facial recognition, verification, and clustering can be easily implemented.

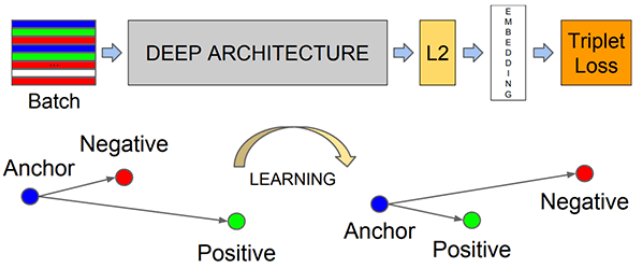


Figure 1: How the deep learning face recognition model computes the face embedding of a figure caption. [5]

Their network consists of a batch input layer and a deep CNN followed by L2 normalization, which results in the face embedding. This is followed by the triplet loss during training.

The Triplet Loss minimizes the distance between an anchor and a positive, both of which have the same identity, and maximizes the distance between the anchor and a negative of a different identity.[5]

To train a face recognition model with deep learning, each input batch of data includes three images such as the anchor, the positive image, and the negative image. The anchor and positive image both belong to the same person/face while the negative image does not contain the same face.

The neural network computes the 128-d embeddings for each face and then tweaks the weights of the network (via the triplet loss function) which results in the 128-d embeddings of the anchor and positive image lie closer together, while at the same time, pushing the embeddings for the negative image father away. By this way, the network can learn to quantify faces and return highly robust and discriminating embeddings suitable for face recognition.

D. Training a support vector(SVM) on top of the embeddings

The next step on the face recognition pipeline is to create and train a machine learning model which can classify and sort the embeddings and give us output for our images. For this we used a support vector model from the ski-learn machine learning library.[6]

The Support Vector Machine (SVM) algorithm is mostly used for pattern recognition. It has been in use since the 1990's and is considered one of the most effective machine learning algorithms. In addition to being applied to face recognition, SVM has been used in other pattern classification applications such as speech recognition, image recognition, text categorization, etc.

Pattern recognition is a powerful tool used in many disciplines for data separation. It uses given parameters or statistical information learned from raw data to classify data. As a type of supervised machine learning, SVM relies on given parameters. It is given a set of training examples pre-sorted into categories; in our case, the training examples are the datasets of facial recognition targets. The SVM training algorithm builds a model based on the given categories to sort a new sample into one of those categories.[7]

E. Recognize faces in images and video streams

After the training of the algorithm, we placed our data and algorithm on Raspberry pi and created 2 modes for operation, and then test the system for efficiency and real-life scenarios.

The images below show a simple overview of method:

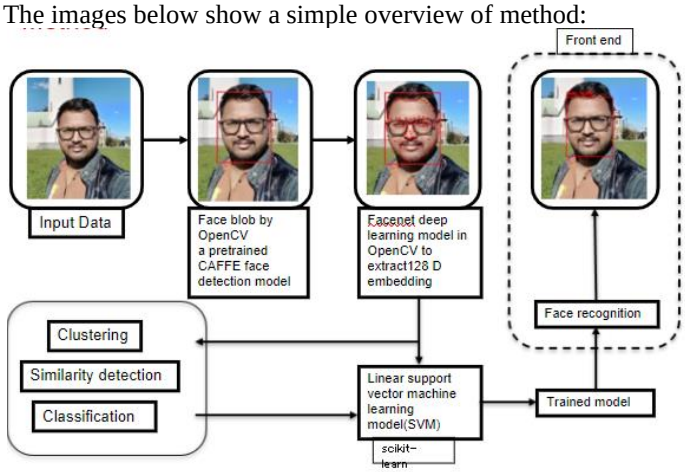


Figure 2: Face recognition pipeline with example

V. PSEUDO CODE

We have written our code in python on OpenCV by taking reference from [15] and in this section we have explained the logic of our code and it is explained as in pseudo-code. This section is divided in three parts as follows:

1) Pseudo Code for Save Dataset of people
2) Pseudo Code for Training face recognition algorithm
3) Pseudo Code for face recognition algorithm

A. Pseudo Code for Save Dataset of people

Our first task is to collect and save images of the target person in our dataset directory. The following is the pseudocode for this task:

```
import all necessary packages

arguments_parser = parse all arguments from
terminal of raspberry pi to this variable

args = save the arguments as an object
using vars()

path=save common pathname and manipulate
pathname to any desirable pathname

load neural network

face_detector=use DNN face detector in OpenCV

face= read the network model stored
in face_detector by passing the arguments (text
descriptor of network architecture,file with
learned network)

face_embeddings= save the face embeddings
through reading the network model stored in the
torch9 framework format by passing the arguments
(model path to file, if network serialized is
ascii or binary, testing phase of network)

picture_path= save the captured images in this
file directory

for loop starting i=0
in picture_path to enumerate(picture_path):
    person_name=save images in respective person
name directory

    person_image=load person image from
directory using OpenCV

    change size of person_image without changing
aspect ratio

    height,width=save person_image dimensions

    person_blob= make blob from
the person_image

    person_detection= use OpenCV deep
learning techinique to familiarize faces
of person_image


if person_detection == person_image detected:

        probability_image=ensure maximum
probability of detecting of each face

        if probability_image == strong detection
of face:

            image_x,image_y= find
the x,y coordinates of the
image withinh bounding box

            person_roi = find the automatic
region of interest of person's face

            roi_x,roi_y= save the region of
interest x and y coordinates

            if roi_x and roi_y == large enough:

                continue

            else:
                break

            roi_blob = find blob of person_roi

            obtain 128-d quantization
of roi_blob

save all face embedding in the respective
directory with respective names of person
```

B. *Pseudo Code for Training face recognition algorithm*

Now after collecting and saving images of target people in their respective directory, we need to train our model:

```
import the necessary packages

arguments_parser = parse all arguments from
terminal of raspberry pi to this variable

args = save the arguments as an object
using vars()

faces_data= save all face data and load all
embeddings

encode face labels with respective names

train the deep learning model with all faces by
```

```
    passing 128-d face embeddings
7.  produce the face recognition model
8.  write this face recognition model for raspberry
    pi disk which includes faces with names
```

C. *Pseudo Code for face recognition algorithm*

Finally, we need to run the face recognition python code on the raspberry pi terminal. The following is its pseudo code:

```
1.  import the necessary packages
2.  button_gpio1=input() #click mode
3.  button_gpio2=input() #continuous mode
4.  face_mobile_application= configure the
    mobile andriod application by
    setting URL, ip address, etc
5.  arguments_parser = parse all arguments from
    terminal of raspberry pi to this variable
6.  args = save the arguments as an object
    using vars()
7.  path=save common pathname and manipulate pathname
    to any desirable pathname
8.  face_detector=use DNN face detector in OpenCV
9.  face= read the network model stored
    in face_detector by passing the arguments (text
    descriptor of network architecture,file with
    learned network)
10. face_embeddings= save the face embeddings through
    reading the network model stored in the torch9
    framework format by passing the arguments (model
    path to file, if network serialized is ascii or
    binary, testing phase of network)
11. open the face recognition model with the encoded
    face labels of respective names
12. open the raspberry pi USB port connected with
    camera
13. live_video=stream video live
14. name_arr=[0,0,0,0,0,0] #this is an array which
    keeps count of how many times a respective person
    was recognized with everyone having their own
    index
    '''
    Index 0: Aneesha or Aneesha Mask
    Index 1: Manas or Manas Mask
    Index 2: Frank or Frank Mask
    Index 3: Rohit or Rohit Mask
    Index 4: Anselm or Anselm Mask
    Index 5: Kshama or Kshama Mask
    '''
15. while True and button_gpio2==pressed:
16.     video_frame=capture video frames
    from live_video.read()
17.     change size of person_image without changing
    aspect ratio
18.     height,width=save person_image dimensions
19.     person_blob= make blob from
    the person_image
20.     person_detection= use OpenCV deep
    learning techinique to familiarize faces
    of person_image
21.     if button_gpio1==pressed:
22.         Change state to click mode
            break
23. if person_detection == person_image detected:
24.         probability_image=ensure maximum
    probability of detecting of each face
25.         if probability_image == strong detection
    of face:
26.             image_x,image_y= find
    the x,y coordinates of the image withinh bounding
    box
27.             person_roi = find the automatic
    region of interest of person's face
28.             roi_x,roi_y= save the region of
    interest x and y coordinates
29.             if roi_x and roi_y == large enough:
                    continue
30.             else:
                    break
31.             roi_blob = find blob of person_roi
32.             obtain 128-d quantization
    of roi_blob
33.             perform necessary steps to classify
    and recognize face
34.             name= save name of person detected in
    live stream by drawing bounding box across frame
    with the maximum probability
35.             if name=="Aneesha" or
    name=="Aneesha_mask":
36.                 name_arr[0]=increment value of
    respective index 0 by 1
37.             else if name=="Manas" or
    name=="Manas_mask":
38.                 name_arr[1]=increment value of
    respective index 1 by 1
39.             else if name=="Frank" or
    name=="Frank_mask":
40.                 name_arr[2]=increment value of
    respective index 2 by 1
41.             else if name=="Rohit" or
    name=="Rohit_mask":
42.                 name_arr[3]=increment value of
    respective index 3 by 1
43.             else if name=="anselm" or
    name="anselm_mask":
44.                 name_arr[4]=increment value of
    respective index 4 by 1
45.             else if name=="kshama" or
    name="kshama_mask":
46.                 name_arr[5]=increment value of
    respective index 5 by 1
47.             initialize firebase by the
    configurations done in face_mobile_application
48.             load the names in firebase server
                #send data to application when a
    person is recognized 3 times minimum
49.          if name_arr[0]==3 or name_arr[1]==3
    or name_arr[2]==3 or name_arr[3]==3
    or name_arr[4]==3 or name_arr[5]==3:
50.             send name to face_mobile_application
51.         else:
52.          send "Detecting Person"
    to face_mobile_application
```

VI. FLOWCHART

Our system can be understood by the flowchart shown in figure 3. The working of our prototype could be understood in the following steps:

1) The Raspberry Pi is turned on, and runs using power supplied from a rechargeable battery.
2) After turning on the Raspberry Pi, we ensure that the USB camera is connected to one of the Pi's several USB ports. We need to read the camera from that respective port.
3) Our prototype works in two different modes: Continuous Mode and Click Mode, as described in Section I of our paper. After reading the camera, the code will check what mode our camera is in. The default mode is continuous mode. But if the "mode" pushbutton is pressed, then the code will switch to the other mode, e.g. Continuous Mode will switch to Click Mode. In click mode, our process would remain in an idle mode, and would wait until the "trigger" button is pressed to begin searching for a face.
4) When the code is searching for a face, the camera sends live video of the environment to the microprocessor to be analyzed.
5) If multiple people are detected, our algorithm would send the message "Detecting person" to our application because our algorithm can only detect one person at a time. So, the individual wearing the clipped camera must look at only one person at a time, and should do so for a minimum of 3 seconds to allow for camera stabilization and face recognition.
6) If one person is detected, then the algorithm begins to search for a match of that person within the stored dataset in the raspberry pi.
7) Our raspberry pi contains a dataset of photos of several individuals. Previously, a training algorithm for recognizing the faces stored in this set of data had been executed, allowing the algorithm to recognize those faces. Now, the algorithm compares the face currently detected with the stored information. If the person gets recognized, then the name would be sent to our phone application and displayed. But if the person is not recognized, then the text "Unknown" is sent to our phone application and displayed.

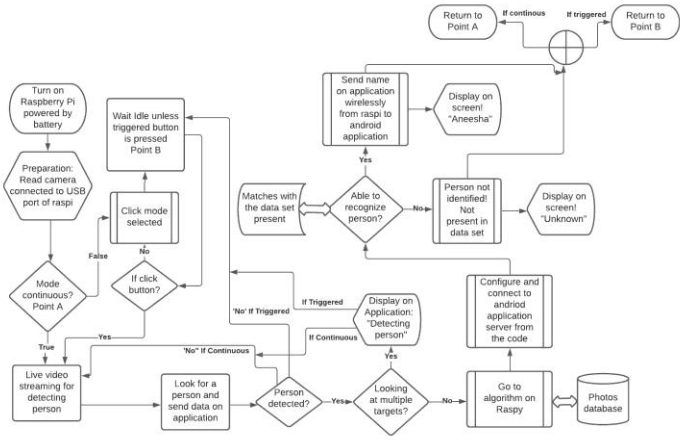


Figure 3: Flowchart of our system

VII. COMPONENTS

There are two categories of components in our project:

1) Auxiliary Components
2) Purchased Components
3) Electrical Components
4) Fabricated Components

A. Auxiliary Components

These components are needed to set up FACE but are not a part of the final build. Once the FACE code is running and a wifi connection is established, all auxiliary components may be unplugged from the Raspberry Pi, and FACE can stand alone.

i. *Computer Monitor:* The Raspberry Pi must be connected to a monitor to interface with the Raspberry Pi operating system.

ii. *Mouse and Keyboard (or laptop with remote access enabled):* These are used to activate the FACE program from the Raspberry Pi operating system, and to establish a wifi connection.

B. Purchased Components

These components were purchased from Amazon and integrated into our product.

i. *Raspberry Pi 4 Kit (Raspberry Pi 4, Pi charger, 32 gb SD card, HDMI cable), $120 at Amazon[10]:* The main component of this kit is the Raspberry Pi 4, a credit-card sized microprocessor which runs a linux-based operating system. Our python FACE program is run on the RasPi4. The kit's 32gb SD card holds the face database accessed by the FACE program, and the kit's HDMI cable and Raspberry Pi charger are used to connect to a monitor for set-up and to charge the Raspberry Pi/battery respectively.



Figure 4: Raspberry Pi

ii. *G007 Sir Gawain USB Camera with cable + USB male-female extension cable, $28 at Amazon[11]:* This small cubic camera with 20mm (0.8") sides is unobtrusive and provides a 1080p video feed to the Raspberry Pi, to which it is connected to by USB cable. The camera has a 140-degree viewing angle and is capable of operating in low-light.

iii. *Pi Sugar 2 Battery Shield, $50 at Amazon[12]:* The Pi Sugar 2 Battery shield, made by Sugar Labs, attaches to the underside of the Raspberry Pi, and is held in

place by 4 screws. No connections other than the screws are needed-when placed properly, the PiSugar2's power supply pins align with the Pi's power input pins and are held in place by the tension provided by the screws. This battery was chosen due to its compact and unobtrusive nature. The battery makers claim the battery can hold a charge for 5 hours of passive use, and 3 hours of intensive use. We found this claim to be true—the battery held its charge through at least 2 hours of testing.



Figure 5: Battery connected to our raspberry pi

C. Electrical Components

A small circuit was designed for Start/Stop and Trigger modes. The system when switched-on works in a continuous capture mode, to switch to trigger capture mode one of the buttons on the breadboard is programmed to do so.
Below are the components used for this circuit.

i. Breadboard
ii. 1k Ohm Resisters
iii. Push Buttons
iv. Male to Female jumper wires

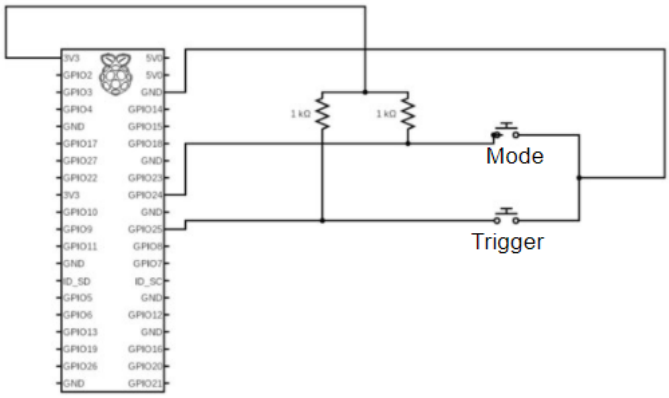


Figure 6: Electrical connecting diagram for our project

D. *Fabricated Components*

These components were designed to carry and protect the functional components of FACE, and were 3D-printed using Northeastern's Interdisciplinary Science & Engineering Complex laboratory printers.

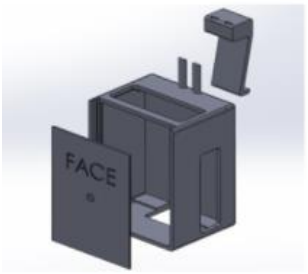
i. *Hip Unit:* The Hip Unit consists of 4 parts: the main body, the belt clip, the belt clip pegs, and the cover.

1.A Hip Unit Main Body: With a profile of 7.62 cm (3") x 9.84 cm (3 7/8"), the main body is slightly larger than the Raspberry Pi. At a depth of 6.05cm (2.38"), the Raspberry Pi has enough vertical clearance to fit along with the Pi Sugar Battery underneath. Clearance is left between the top of the Raspberry Pi and the cover to allow for air circulation and to account for wiring. Appropriately sized slots are present in the sides of the hip unit's main body to allow access to the Raspberry Pi's ports. The main body has a wall thickness of 0.32 cm (1/8"), giving it a sturdy and robust build.

1.B. Hip Unit Belt Clip: The belt clip allows the user to attach the hip unit to the user's belt and is connected to the Main Body via the belt clip pegs. It is 2.5 cm (1") wide, about 6 cm (2.36)" long, and 1.9 cm (0.75") deep.

1.C Hip Unit Belt Clip Pegs: The belt clip pegs connect the belt clip to the main body. Slots are present in both the main body and belt clip, and the pegs, precision-designed along with the slots, firmly connect the clip and main body through friction. The pegs have a semicircular profile with a radius of 0.32 cm (1/8"), and are 3.8 cm (1.5") long.

1.D Hip Unit Cover: The cover is simply pressed into place on the top of the main body and is held in place through friction.



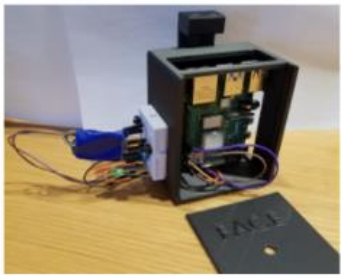


Figure 7: Solid works model of hip unit *Figure 8: Hip unit after 3d print*

ii. *Glasses Clip:* The glasses clip is a single unit, consisting of a camera frame and a glasses clip. The camera frame is sized to be the same width as the G007 Sir Gawain camera, about 2 cm (0.8"), but has a height 10% smaller than the G007. The plastic has enough give to ensure a firm grasp of the camera. The two sides of the glasses clip have 3.175mm (1/8") clearance between them and are covered with a series of pegs. The pegs come in pairs and have varying clearances to account for different glasses frame heights. The pair clearances vary from 12.7mm (1/2") to 3.175mm (1/8") and decrease in increments of 1.588mm (1/16") moving up the length of the clip, alternating sides. The clip is about 5.1 cm (2") tall and 2.54 cm (1") deep, allowing it to keep a modest profile.



Figure 9: Camera clipper after 3d print with G007 camera



Figure 10: Solid works model of camera clipper

VIII. SYSTEM ARCHITECTURE

The system architecture is classified into the following parts:

1) Image capture and classification
2) Database
3) Facial Detection
4) Firebase Server Connectivity
5) FACE Android Application

A. Image capture and classification

Images of the desired target person are captured using the G007 Sir Gawain camera. These images are taken in multiple angles, different lighting scenarios, diverse backgrounds, and, for target persons wearing glasses, images are captured with and without glasses.

Considering that our protype is made to work in an indoor scenario, facial detection with and without masks is very crucial as, given the ongoing Covid-19 pandemic, people are expected to wear masks indoors. Thus, images of the target person are captured with and without masks with all the pre-mentioned conditions.

Once the required number of images are taken, these images are then classified according to the target person's name and stored in the database.

B. Database

The captured images, once classified, are then stored in the local database on the Raspberry Pi microprocessor. This database is accessible by Raspberry Pi throughout the course of execution.

This database is completely modifiable, i.e., various new datasets can be added, modified or even removed from the database as and when seem fit.

C. Facial Detection

The G007 Sir Gawain camera is used for facial detection. The live stream from the camera is given as an input for the Raspberry Pi. Here the Raspberry Pi makes use of the facial recognition algorithm built, through this identifies the faces as detected in the live video stream and compares it with the datasets available in the database.

D. Firebase Server Connectivity

The output from the Raspberry Pi is sent to a unique Firebase Server created for the FACE project. This server is used for connectivity between the Raspberry Pi and the exclusive FACE android app created.

This is a real-time server that has 2 status modes. "Status 1" for if there is a target detected and a name that is to be displayed. "Status 2" if there is no detection.

E. FACE Android Application

We have created an exclusive app in the Android Platform for the FACE Project which acts as an output screen for the user when using the prototype. The App has 2 main screens. Screen 1 – is the starting screen with Name, Logo, and the motto for the project. Screen 2 – is the main detection screen where the outputs are printed.

IX. SOFTWARE IMPLEMENTATION

In our prototype, the software has been implemented in two ways. The first is what we are doing and how are we configuring everything on the raspberry pi unit. This part is hidden from the user. The second is what steps the user must take to get the application working properly. We have explained both the implementations in the following subsections:

1) Software Implementation I
2) Software Implementation II

A. Software Implementation I

In the backend part of our application the following steps are being executed which the user would be unaware of:

i. *Installing libraries and integrating Raspberry Pi:*

We install several libraries on the raspberry pi order to get our program working and to perform the face detection algorithm successfully. After installing them, we connect the raspberry pi to the intended-based server, which in our project is firebase (discussed in the next subsection in detail). This is done so that the raspberry pi can send data to the application server wirelessly.

ii. *Create dataset:*

After configuring the raspberry pi and compiling our OpenCV python code, we need to collect data of the people the user wants to have on his/her dataset. To collect database images, we run our image collection code on a computer, and have the target look at a camera connected to the computer running the code. The target captures an image and saves their images in the raspberry pi directory by hitting the space button on the laptop. The target should vary their profile in their images; variations can include staring into the camera, looking in some different directions, turning their head to present a side profile, with and without mask, and so on. The more images we have, the better the algorithm would execute. We should collect a minimum of 50 images with and without a mask.

iii. *Train our algorithm:*

By running a different code on the raspberry pi terminal, we can train our algorithm, what this means is that the machine learning algorithm would be run and implemented on the raspberry pi. Up to this point, we have been working on our code via computer due to ease of use. Training our algorithm with a database on the Raspberry Pi is a time-intensive process, especially if the user wants to account for many people, requiring a greater number of images to be held in the data. However, it is absolutely necessary to train the algorithm, because without the training the target person will not be detected.

iv. *Run face-detection algorithm on Raspberry Pi:*

Now we will run the application on the raspberry pi terminal. This will activate the camera, and then our entire algorithm will be ready for detecting a person.

B. Software Implementation II

i. *Firebase Server:* Firebase is developed by Google to give developers a good platform to develop high quality mobile applications. We are making use of this platform for our mobile app to convey the output message from the Raspberry Pi during the execution of our algorithm in the prototype. The overall set up in the server is a simple operation with one node operating with 2 statuses:

a) ***'Status 1'*** - This is the status that is triggered when there is a detection of the target person in the live stream. In the case of any detection or if a target person is being detected or if an unknown target is detected is when this Status is implemented.
b) ***'Status 0'*** - This status is triggered when there is no detection, i.e., there is no target available for detection at all in the live video stream.

ii. On the Raspi, 'Pyrebase' - a python wrapper for Firebase API is installed to enable ther communcation between Raspberry Pi and the Firebase Server. [8]

iii. *Android App:* We used Flutter to design a simple but intuitive mobile application exclusively for the FACE project for the android platform. The Application is used as a display for the output data sent by Raspberry Pi. This Application is connected to the Raspberry Pi through the Firebase server created. There are 4 outputs that are displayed on this screen.

a) ***Name of the person detected*** – If the target person is detected then the name of the target is presented on this screen.
b) ***'Detecting person'*** - This is the output on the screen when the prototype is trying to identify the target amongst a group of people present in the camera view.
c) ***'Unknown'*** - This is displayed on the screen when the target is not identified by the prototype.
d) ***'No Detection'*** - This is the output when the prototype does not detect any target person in the view of the camera.

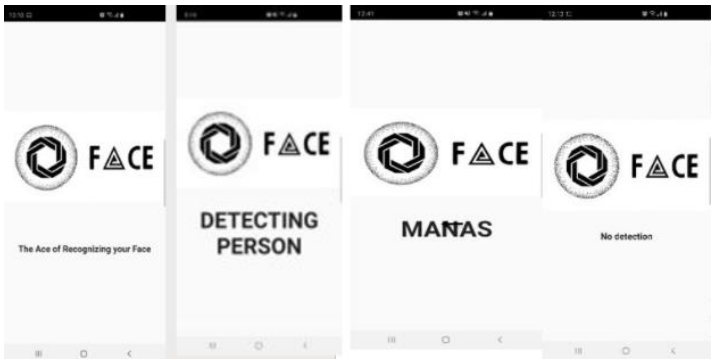


Figure 11: Screen shot of application on starting the application, when detecting a person, person detected, and if no detection respectively

X. RISK MITIGATION

We identified few risks associated with our project and the following is the risks we identified along with how could we mitigate them:

1) Luminosity is one of the many factors affecting the performance of the camera. Low or high luminosity affects the precision of the target capture algorithm, and can result in false identification of the target, or not identifying the target altogether. As a prototype, we are limiting this product to indoors with decent lighting for better target capture and data processing.
2) Data accuracy can be improved by training the system with as many data sets as possible. As a risk mitigation plan the system was trained with 80 images per person, for 6 people – which totals up to 480 images in all. The more data in the dataset, the better. We were able to achieve moderate accuracy with the limited data sets and time available.
3) The G007 Sir Gawain camera takes time to adjust the focus. We are implementing a 3-second delay for the camera to stabilize and focus on the target to capture a better image.
4) The microprocessor is the heart of the project. The project is making use of Raspberry Pi for all the computation. It has proven to be sufficient with satisfactory results. As a back-up we were considering Jetson Nano as a microprocessor to drive our project but did not find it necessary to use it.
5) Mask detection was a hurdle with a high chance of failure. It was taken care of by training the system with more data sets per person. Training the system is a time-consuming process, but an expected accuracy of detecting the target with a mask and without a mask was achieved.
6) The captured image is matched with the dataset and the output is sent to the firebase which is later updated on the android application. The project is using Wi-Fi module built in on the Raspberry Pi.

XI. LIMITATIONS

The project has some limitations as discussed:

1) The Raspberry Pi processor is introducing some delay as it has 4GB RAM, which is used up by the

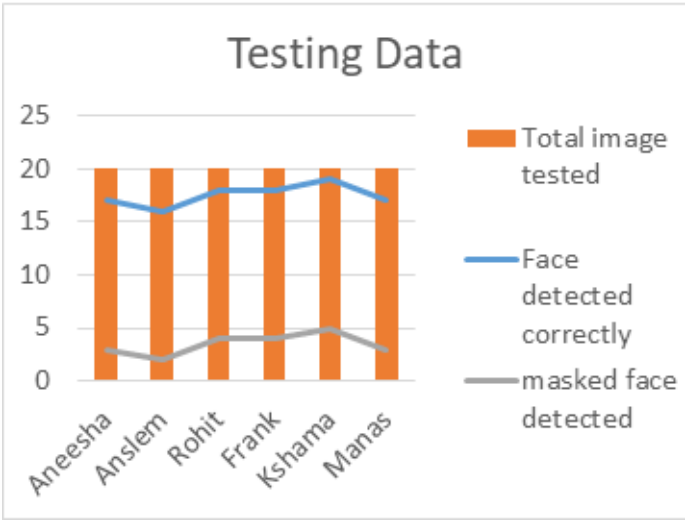
algorithm, and the datasets causing a reduced operating speed, i.e., creating a lag.

2) The camera attached to the head unit tends to tilt to one side due to the apparent weight of the clip holding the camera and the wiring on one side. We suggest overcoming this by using counterweights on the other side of the glass, using lighter cables.
3) Not only the user end but the target also needs to satisfy certain conditions. The target must look directly into the camera, the height of the target needs to be in the line of sight of the camera. The camera fails to identify the target in low or high luminosity conditions, the target must be positioned in a decent lighting.

XII. ACCURACY ANALYSIS

For accuracy and efficiency, we took 20 images from each person and ran them through our system out of which most of the results were good.

We can observer the graph below for specific set of people and their correct face detection frequency:



Graph 1: Plotting of faces detected and faces not detected

With the above figures in mind, we go 87.5 percent accuracy with still images. This result is also depended on the tested image as most of the image were taken with straight orientation, which is beneficial to the face recognition system.
For masked face detection testing we took 5 images per person, and we got an average of 3 to 4 detection per person overall with accuracy of 70 percent.

For video-based testing we randomly took 5 min of footage and checked how many faces it recognized correctly. In the first few seconds after the camera is first turned towards a person, face recognition is mostly inaccurate due to the camera lag, and camera movement and resulting blur. However, after a few seconds of stabilization we were able to reach 70 percent efficiency, as we also considered pre stabilization faces in checking the algorithm.

The biggest limitation of this system is face alignment; as the target face tilts away from looking straight on at the camera, the accuracy of the model decreases.

XIII. CONCLUSION

Though undoubtedly still a prototype, FACE lives up well to its design requirements. FACE's basic operation is simple and intuitive. When active, the user only interfaces with FACE through two buttons on the side and reads output through a basic web app. Activating the Raspberry Pi and executing the FACE program are slightly more complex tasks, but are well within the abilities of even novice Raspberry Pi users. Likewise, updating FACE's facial database only requires the addition of more files and the execution of our analysis program. Due to the large database of user experiences available for the Raspberry Pi, troubleshooting or modifying code are also readily attainable tasks.

Although still sensitive to environmental conditions such as lighting, and slight variations in target presentation such as head tilt, FACE is able to detect and recognize the target subjects. After future code optimizations, or with a more varied database, accuracy should increase significantly. Additionally, the device is highly portable; both its glasses camera clip and hip unit are small, sturdy, and lightweight. Lastly, FACE is cost-effective at a price of around $200, increasing accessibility. After taking all variables into consideration, we can conclude that FACE may well be the ace of recognizing your face.

XIV. REFERENCES

[1] Grüter T, Grüter M, Carbon CC (2008). "Neural and genetic foundations of face recognition and prosopagnosia". J Neuropsychol. 2 (1): 79–97. CiteSeerX 10.1.1.571.9472. doi:10.1348/174866407X231001. PMID 19334306.

[2] Yardley L., McDermott L., Pisarski S., Duchaine B., Nakayama K. (2008). Psychosocial consequences of developmental prosopagnosia: a problem of recognition. J Psychosom. Res. 65, 445–451 10.1016/j.jpsychores.2008.03.013J.

[3] DeGutis JM, Chiu C, Grosso ME, Cohan S. Face processing improvements in prosopagnosia: successes and failures over the last 50 years. Front Hum Neurosci. 2014;8:561. Published 2014 Aug 5. doi:10.3389/fnhum.2014.00561

[4] Tirta Susilo, Bradley Duchaine,Advances in developmental prosopagnosia research,Current Opinion in Neurobiology,Volume 23, Issue 3,2013,Pages 423-429,ISSN 0959-4388, https://doi.org/10.1016/j.conb.2012.12.011.

[5] Schroff, D. Kalenichenko and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 815-823, doi: 10.1109/CVPR.2015.7298682.

[6] Pedregosa, F. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research : JMLR., 12.

[7] Pradhan, Ashis. "Support vector machine-a survey." International Journal of Emerging Technology and Advanced Engineering 2.8 (2012): 82-85.

[8] James Child-Maidment, “Pyrebase”, https://github.com/thisbejim/Pyrebase, 2020.

[9] M. Young, The Technical Writer’s Handbook. Mill Valley, CA: University Science, 1989.

[10] https://www.amazon.com/dp/B08B6G2RFG/ref=cm_sw_em_r_mt_dp_6ME2YT96JYEWJTA1099J?_encoding=UTF8&psc=1
[11] https://www.amazon.com/Mini-Camera-1080P-Hidden-Built/dp/B07X1YLC2L/ref=psdc_12909791_t1_B07GCKZKX8
[12] https://www.amazon.com/dp/B08D8PPCKN?tag=bgfg-pcguide-us-20&geniuslink=true
[13] https://www.orcam.com/en
[14] https://www.vuzix.com/products/m4000-smart-glasses
[15] Rosebrock Adrian, OpenCV Face Recognition,pyimagesearch.com,September 24, 2018,https://www.pyimagesearch.com/2018/09/24/opencv-face-recognition/